Sequential Circuits Tutorial

Table of Contents

Introduction

Sequential circuits are digital logic circuits whose outputs depend not only on the current inputs but also on the previous states (history). Unlike combinational circuits, they have memory capability.

Key Characteristics:

- **Memory Elements**: Store previous state information
- **Feedback Loops**: Output feeds back to input through memory elements

- **Clock Dependency**: Most sequential circuits are synchronous (clock-driven)
- **State-Based**: Current output depends on current input AND current state
- **Temporal Behavior**: Output changes with time and clock cycles

Comparison: Combinational vs Sequential

Feature	Combinational	Sequential
Memory	No memory	Has memory
Output depends on	Current inputs only	Current inputs + past state
Timing	Asynchronous	Usually synchronous
Feedback	No feedback	Has feedback paths
Examples	Gates, MUX, Decoder	Flip-flops, Counters, FSMs

This tutorial was generated with Claude AI(accessed: 2025-09-08).
Modified and reviewed by JACH for CMSC 132.

Latches

Latches are basic memory elements that can store one bit of information. They are **level-triggered** (sensitive to logic levels).

1. SR Latch (Set-Reset Latch)

Using NOR Gates:

- **Inputs**: S (Set), R (Reset)
- **Outputs**: Q, $\bar{Q}$ (complement of Q)

Truth Table:

S	R	Q(t+1)	$\bar{Q}$(t+1)	Action
0	0	Q(t)	$\bar{Q}$(t)	Hold (No change)
0	1	0	1	Reset
1	0	1	0	Set
1	1	0	0	**Invalid/Forbidden**

Characteristic Equations: - $Q(t+1) = S + \bar{R} \cdot Q(t)$ - Invalid when $S = R = 1$

Using NAND Gates ($S\bar{R}$ Latch):

- **Inputs**: $\bar{S}$ (active-low Set), $\bar{R}$ (active-low Reset)
- **Invalid state**: $\bar{S} = \bar{R} = 0$

2. Gated SR Latch

- **Inputs**: S, R, Enable (E)
- **Operation**: SR latch is active only when Enable = 1
- **Truth Table**: Same as SR latch, but only when E = 1

Logic: - Internal S = S_input · E - Internal R = R_input · E

3. D Latch (Data Latch)

- **Purpose**: Eliminates invalid state of SR latch
- **Inputs**: D (Data), Enable (E)
- **Operation**: Q follows D when Enable = 1

Truth Table:

D	E	Q(t+1)	Action
X	0	Q(t)	Hold
0	1	0	Store 0
1	1	1	Store 1

Implementation: D connects to S, $\bar{D}$ connects to R of gated SR latch

4. JK Latch

- **Inputs**: J, K, Enable (E)
- **Advantage**: No invalid states (unlike SR latch)

Truth Table:

J	K	E	Q(t+1)	Action
X	X	0	Q(t)	Hold
0	0	1	Q(t)	No change
0	1	1	0	Reset
1	0	1	1	Set
1	1	1	$\bar{Q}$(t)	Toggle

Flip-Flops

Flip-flops are **edge-triggered** memory elements (sensitive to clock transitions). They are more reliable than latches for synchronous systems.

1. D Flip-Flop (Data/Delay Flip-Flop)

Positive Edge-Triggered:

- **Inputs**: D (Data), CLK (Clock)
- **Operation**: Q changes to D value on positive clock edge (0→1)

Truth Table:

D	CLK	Q(t+1)	Action
X	0 or 1 (no edge)	Q(t)	Hold
0	↑ (rising edge)	0	Store 0
1	↑ (rising edge)	1	Store 1

Characteristic Equation: Q(t+1) = D

This tutorial was generated with Claude AI(accessed: 2025-09-08).
Modified and reviewed by JACH for CMSC 132.

With Asynchronous Inputs:

- **Preset ($P\bar{R}$)**: Sets Q = 1 immediately (active low)
- **Clear ($CL\bar{R}$)**: Resets Q = 0 immediately (active low)
- These inputs work regardless of clock

2. JK Flip-Flop

- **Inputs**: J, K, CLK
- **Advantage**: No invalid states, includes toggle function

Truth Table:

J	K	CLK	Q(t+1)	Action
X	X	No edge	Q(t)	Hold
0	0	↑	Q(t)	No change
0	1	↑	0	Reset
1	0	↑	1	Set
1	1	↑	$\bar{Q}$(t)	Toggle

Characteristic Equation: $Q(t+1) = J \cdot \bar{Q}(t) + \bar{K} \cdot Q(t)$

3. T Flip-Flop (Toggle Flip-Flop)

- **Input**: T (Toggle), CLK
- **Operation**: Toggles output when T = 1

Truth Table:

T	CLK	Q(t+1)	Action
X	No edge	Q(t)	Hold
0	↑	Q(t)	No change
1	↑	$\bar{Q}$(t)	Toggle

Characteristic Equation: $Q(t+1) = T \cdot \bar{Q}(t) + \bar{T} \cdot Q(t) = T \oplus Q(t)$

Implementation: JK flip-flop with J = K = T

4. SR Flip-Flop

- Similar to SR latch but edge-triggered
- **Invalid state**: S = R = 1 still forbidden

This tutorial was generated with Claude AI(accessed: 2025-09-08).
Modified and reviewed by JACH for CMSC 132.

Timing Concepts

1. Setup Time (tsu)

- **Definition**: Minimum time data must be stable BEFORE clock edge
- **Violation**: Data changes too close to clock edge → unpredictable output

2. Hold Time (th)

- **Definition**: Minimum time data must remain stable AFTER clock edge
- **Violation**: Data changes too quickly after clock → unpredictable output

3. Clock-to-Q Delay (tcq)

- **Definition**: Time from clock edge to output change
- **Types**:
  - **tpLH**: Propagation delay Low-to-High
  - **tpHL**: Propagation delay High-to-Low

4. Maximum Frequency (fmax)

- **Formula**: fmax = 1 / (tcq + tpd + tsu)
- **Where**: tpd = propagation delay through combinational logic

5. Metastability

- **Cause**: Setup/hold time violations
- **Effect**: Output oscillates or settles to intermediate voltage
- **Prevention**: Use synchronizers, proper timing design

Registers

Registers are collections of flip-flops used to store multi-bit data.

1. Parallel Load Register

- **Purpose**: Store n-bit data simultaneously
- **Components**: n D flip-flops with common clock
- **Operation**: All bits loaded on same clock edge

This tutorial was generated with Claude AI(accessed: 2025-09-08).
Modified and reviewed by JACH for CMSC 132.

4-bit Parallel Load Register:

- **Inputs**: D3, D2, D1, D0, CLK, Load
- **Outputs**: Q3, Q2, Q1, Q0
- **Operation**: When Load = 1, Q = D on clock edge

2. Shift Registers

Store data and shift it left or right on each clock cycle.

Types:

1. **Serial-In Serial-Out (SISO)**
2. **Serial-In Parallel-Out (SIPO)**

3. **Parallel-In Serial-Out (PISO)**
4. **Parallel-In Parallel-Out (PIPO)**

Universal Shift Register:

- **Capabilities**: Parallel load, shift left, shift right, hold
- **Control**: Mode select inputs determine operation

Mode Control:

S1	S0	Operation
0	0	Hold
0	1	Shift Right
1	0	Shift Left
1	1	Parallel Load

3. Applications of Registers:

- **Data Storage**: Temporary storage in processors
- **Serial Communication**: UART, SPI protocols
- **Arithmetic Operations**: Multiplication, division
- **Delay Lines**: Digital signal processing

Counters

Counters are sequential circuits that go through predetermined sequences of states.

This tutorial was generated with Claude AI(accessed: 2025-09-08).
Modified and reviewed by JACH for CMSC 132.

1. Asynchronous Counters (Ripple Counters)

Binary Ripple Counter:

- **Structure**: T flip-flops in cascade
- **Clock**: Only first FF gets external clock
- **Operation**: Each FF toggles when previous FF goes 1→0

4-bit Binary Counter Sequence:

```
Clock | Q3 Q2 Q1 Q0 | Decimal
------|-------------|--------
  0   |  0  0  0  0 |    0
  1   |  0  0  0  1 |    1
  2   |  0  0  1  0 |    2
  3   |  0  0  1  1 |    3
  ...
 15   |  1  1  1  1 |   15
 16   |  0  0  0  0 |    0 (repeats)
```

Disadvantages: - **Ripple Delay**: Cumulative delay through all stages - **Glitches**: Temporary incorrect outputs during transitions

2. Synchronous Counters

- **Clock**: Common clock to all flip-flops
- **Advantages**: No ripple delay, glitch-free
- **Design**: More complex logic required

4-bit Synchronous Binary Counter:

- **T0 = 1** (always toggles)
- **T1 = Q0** (toggles when Q0 = 1)
- **T2 = Q1·Q0** (toggles when Q1 = Q0 = 1)
- **T3 = Q2·Q1·Q0** (toggles when Q2 = Q1 = Q0 = 1)

3. Decade Counter (MOD-10)

- **Count Sequence**: 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 0, ...
- **Implementation**: 4-bit counter with additional logic to reset at 10

4. Up/Down Counter

- **Up Mode**: Normal binary counting (increment)
- **Down Mode**: Reverse counting (decrement)
- **Control**: Up/Down control input

This tutorial was generated with Claude AI(accessed: 2025-09-08).
Modified and reviewed by JACH for CMSC 132.

5. Programmable Counters

- **Feature**: Can be loaded with any initial value
- **Inputs**: Parallel load inputs, Load enable
- **Applications**: Divide-by-N counters, timing circuits

State Machines

Finite State Machines (FSMs) are sequential circuits designed to follow specific state sequences based on inputs.

1. Components of FSM:

- **States**: Distinct conditions of the system
- **Transitions**: Changes between states based on inputs
- **Outputs**: Functions of current state (and inputs)

2. Types of FSMs:

Moore Machine:

- **Output**: Depends only on current state
- **Advantage**: Output stable throughout clock cycle
- **Disadvantage**: May require more states

Mealy Machine:

- **Output**: Depends on current state AND current input
- **Advantage**: Fewer states possible
- **Disadvantage**: Output may change during clock cycle

3. FSM Design Steps:

1. **Problem Statement**: Define requirements clearly
2. **State Diagram**: Draw states and transitions
3. **State Table**: Tabulate next states and outputs
4. **State Assignment**: Assign binary codes to states
5. **Logic Minimization**: Derive simplified Boolean expressions
6. **Implementation**: Build circuit using flip-flops and logic gates

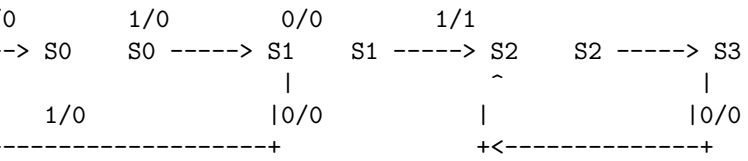
4. Example: 3-bit Sequence Detector (101)

Problem: Detect sequence “101” in serial input stream

States: - S0: Initial state (no progress) - S1: Received “1” - S2: Received “10” - S3: Received “101” (output = 1)

This tutorial was generated with Claude AI(accessed: 2025-09-08).
Modified and reviewed by JACH for CMSC 132.

State Diagram:



Memory Elements

1. Static RAM (SRAM)

- **Structure**: Cross-coupled inverters (6 transistors per bit)
- **Characteristics**: Fast, volatile, expensive
- **Applications**: Cache memory, register files

2. Dynamic RAM (DRAM)

- **Structure**: Capacitor + transistor per bit
- **Characteristics**: Slower, needs refresh, cheap, high density
- **Applications**: Main memory

3. Read-Only Memory (ROM)

- **Types**: ROM, PROM, EPROM, EEPROM, Flash
- **Applications**: Firmware, lookup tables, microcode

4. Content Addressable Memory (CAM)

- **Function**: Search by content rather than address
- **Applications**: Cache tags, routing tables

Design Methodology

1. Synchronous Design Guidelines:

- **Single Clock Domain**: Use one clock throughout design
- **Registered Outputs**: All outputs should come from flip-flops
- **No Combinational Feedback**: Avoid combinational loops
- **Setup/Hold Margins**: Ensure timing requirements are met

2. Clock Distribution:

- **Clock Skew**: Difference in clock arrival times
- **Clock Tree**: Balanced distribution network
- **Clock Gating**: Disable clocks to reduce power

This tutorial was generated with Claude AI(accessed: 2025-09-08).
Modified and reviewed by JACH for CMSC 132.

3. **Reset Strategy:**

   - **Asynchronous Assert**: Reset immediately when asserted
   - **Synchronous Deassert**: Release reset synchronously
   - **Reset Tree**: Proper reset distribution

4. **Power Considerations:**

   - **Clock Gating**: Reduce dynamic power
   - **Power Islands**: Multiple voltage domains
   - **Sleep Modes**: Turn off unused circuits

Real-World Applications

1. **Microprocessors:**

   - **Program Counter**: Points to next instruction
   - **Registers**: Store operands and results
   - **Pipeline Registers**: Between pipeline stages
   - **Cache Controllers**: Manage cache operations

2. **Communication Systems:**

   - **UART**: Serial communication controller
   - **Protocol Processors**: Handle communication protocols
   - **FIFOs**: Buffer data between different clock domains

3. **Digital Signal Processing:**

   - **Filters**: Digital filter implementations
   - **FFT Processors**: Fast Fourier Transform
   - **Delay Lines**: Signal delays for processing

4. **Control Systems:**

   - **Motor Controllers**: PWM generation, speed control
   - **Industrial Automation**: State machine based controllers
   - **Automotive Electronics**: Engine control, safety systems

5. **Memory Controllers:**

   - **DRAM Controllers**: Manage DRAM refresh and access
   - **Cache Controllers**: Implement cache policies
   - **Memory Arbiters**: Manage multiple memory requestors

This tutorial was generated with Claude AI(accessed: 2025-09-08).
Modified and reviewed by JACH for CMSC 132.

Practice Problems

Problem 1: Counter Design

Design a synchronous counter that counts in the sequence: 0, 2, 5, 6, 0, 2, 5, 6, ...

Requirements: - Use JK flip-flops - Draw state diagram - Create state table - Derive Boolean expressions

Problem 2: Sequence Detector

Design a Moore machine that outputs '1' when it detects the sequence "1011" in a serial input stream.

Requirements: - Draw state diagram - Create state table with state assignments - Implement using D flip-flops

Problem 3: Traffic Light Controller

Design a traffic light controller for a 4-way intersection.

States: North-South Green (30s), NS Yellow (5s), East-West Green (25s), EW Yellow (5s)

Requirements: - Use a counter for timing - Emergency override input - Pedestrian crossing request

Problem 4: UART Transmitter

Design a UART transmitter that sends 8-bit data with 1 start bit and 1 stop bit.

Requirements: - Shift register for parallel-to-serial conversion - State machine for protocol control - Baud rate generation

Advanced Topics

1. **Clock Domain Crossing:**
   - **Synchronizers**: Prevent metastability
   - **FIFO Buffers**: Handle rate differences
   - **Handshaking**: Ensure reliable data transfer

2. **Pipeline Design:**
   - **Pipeline Registers**: Store intermediate results
   - **Hazard Detection**: Handle data dependencies
   - **Branch Prediction**: Reduce pipeline stalls

This tutorial was generated with Claude AI(accessed: 2025-09-08).
Modified and reviewed by JACH for CMSC 132.

3. Low Power Techniques:

- **Clock Gating**: Conditional clock distribution
- **Power Gating**: Turn off power to unused blocks
- **Voltage Scaling**: Dynamic voltage adjustment

4. Testability:

- **Scan Chains**: For manufacturing test
- **BIST**: Built-in self-test
- **Design for Debug**: Observability and controllability

Key Takeaways

1. **Sequential circuits have memory** - output depends on past states
2. **Timing is critical** - setup/hold times must be satisfied
3. **Flip-flops are preferred over latches** for synchronous design
4. **State machines provide structured design** methodology
5. **Registers and counters are building blocks** for larger systems
6. **Clock distribution affects performance** and power consumption
7. **Proper reset strategy is essential** for reliable operation

---

This tutorial was generated with Claude AI(accessed: 2025-09-08).
Modified and reviewed by JACH for CMSC 132.